PRACTICAL- 12

Event Management CRUD with Poster Upload and File Validation

AIM / Problem Definition: Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.

Tools / Technology Used: PHP 8.x, MySQL / phpMyAdmin, Apache / XAMPP, HTML5, CSS3, JavaScript (Fetch / FormData), File Upload API

1. Project Implementation & Event CRUD Architecture

1.1 Project Overview (StudentHub — Campus Event Management)

Following the implementation of the Student Registry Management CRUD system in Practical 11, Practical 12 develops the comprehensive Event Management and Poster Media Upload subsystem for the StudentHub University Portal. This enterprise module enables institutional administrators to publish, moderate, update, and remove campus symposia, hackathons, academic conferences, and cultural sports events. Central to this practical is the secure handling of binary media uploads via PHP multipart/form-data handling. The module enforces strict client and server-side file type and file size validations, implements cryptographically safe file storage and image path association in MySQL, synchronizes newly published events with both the administrative control center and student-facing exploration catalogs, and ensures atomic cleanup of file system assets during record deletion.

1.2 Multi-Tier Architecture & File Upload Security Workflow

The event management module implements defensive file-upload engineering, relational database updates, and asynchronous client communication:

- **Multipart Form Submission (POST):** Event details and binary image files are transmitted via multipart/form-data using the JavaScript FormData API. This permits concurrent transfer of text fields (title, category, date, venue, capacity) alongside the poster image payload without full page refreshes.
- **Strict Server-Side File Type Validation:** File validation never relies solely on the client-supplied file extension or MIME header. Instead, PHP inspects the actual binary stream using finfo_open(FILEINFO_MIME_TYPE) or getimagesize(). Uploads are strictly restricted to verified image MIME types: 'image/jpeg', 'image/png', 'image/webp', and 'image/gif'. Any unsupported file (e.g., .pdf, .docx, .exe) is immediately rejected with an explicit error alert.
- **File Size Enforcement (Max 2MB):** Uploads enforce a strict ceiling of 2 Megabytes (2,097,152 bytes) checked against $_FILES['poster']['size']. Files exceeding this threshold are trapped before disk writing, preventing server storage exhaustion and denial-of-service (DoS) vectors.
- **Cryptographic Filename Sanitization:** To eliminate directory traversal attacks and prevent filename collisions, original filenames are discarded. The backend generates unique timestamped filenames using bin2hex(random_bytes(8)) and microtime (e.g., 'event_poster_20261018_124842_c3532f.jpg'). Files are stored in a dedicated, protected directory ('uploads/events/'), while only the relative URI is stored in the MySQL database.
- **End-to-End CRUD Lifecycle:** The system supports complete administrative operations: Create (INSERT with poster upload), Read (SELECT with visual thumbnails, category filtering, and status badges), Update

(modifying dates, locations, or replacing posters), and Delete (permanently removing records with confirmation guards and unlinking media files from disk).

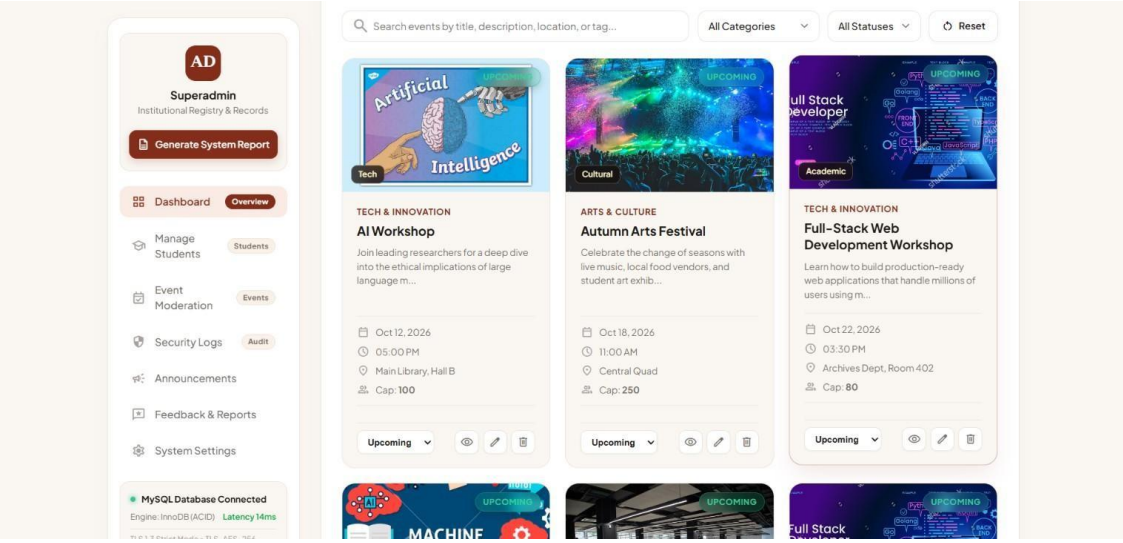
- **Cross-Portal Synchronization:** Newly published events immediately synchronize across the architecture: appearing in the Admin Event Moderation Center for attendance governance and dynamically reflecting in the public Student Catalog ('events.html') with real-time 'Register Now' access.

2. Event Management CRUD & File Validation Outputs

Below are the verified output screens of the StudentHub administrative and student portals demonstrating event browsing, creation with poster upload, file type validation, database persistence, event updates, and deletion:

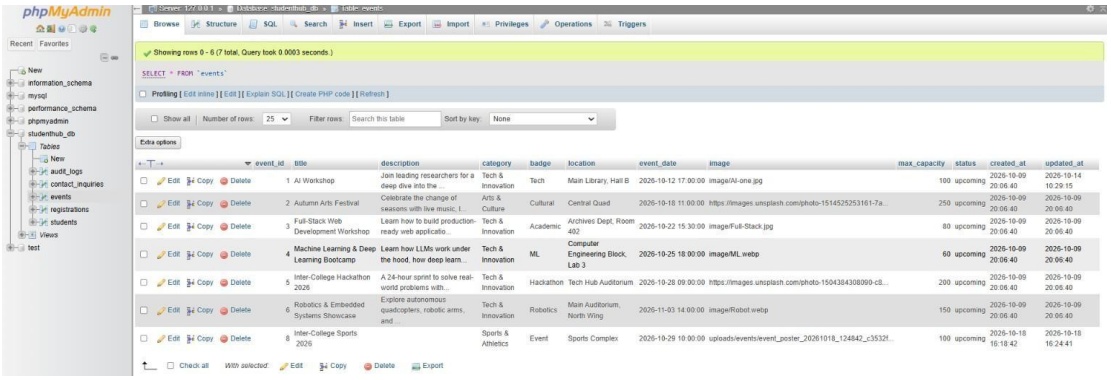
2.1 Output 1: Event Moderation Governance Dashboard (SELECT & Catalog View)

The Event Moderation Control Center within the administrative portal displaying existing campus events in dynamic card layouts. Each card renders the uploaded event banner poster, category pill ('TECH & INNOVATION', 'ARTS & CULTURE'), event title, scheduled date and time, venue location, capacity limit, lifecycle status dropdown ('Upcoming'), and administrative action controls (View, Edit, Delete). Top navigation includes search and category filters.



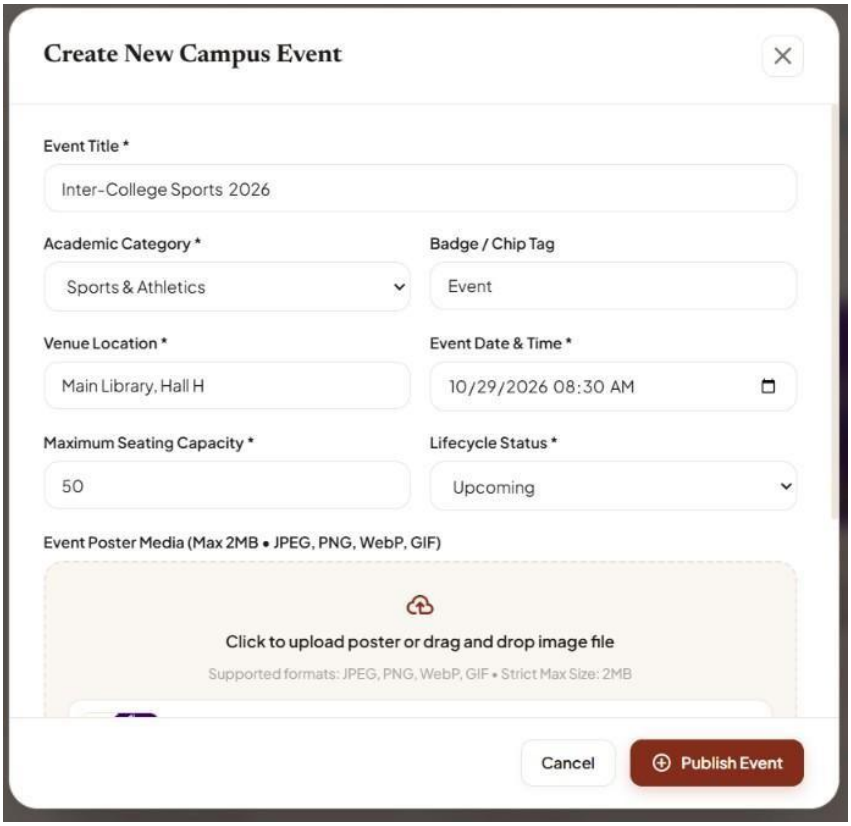
2.2 Output 2: Initial MySQL Database State (`events` Table)

phpMyAdmin database browse view of the 'events' table showing the initial 6 seeded campus events (AI Workshop, Autumn Arts Festival, Full-Stack Web Development Workshop, Machine Learning Bootcamp, Inter-College Hackathon 2026, Robotics Showcase) with their associated poster image paths and upcoming lifecycle statuses.



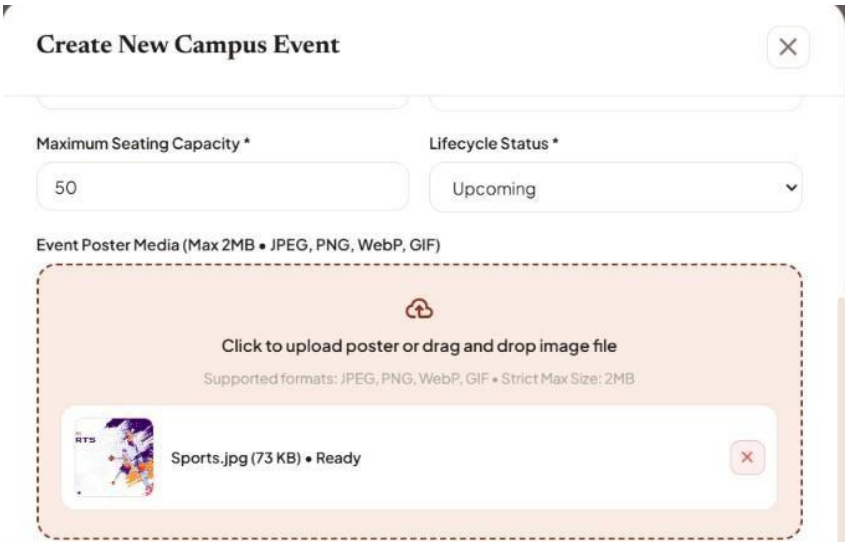
2.3 Output 3: Create New Campus Event Form (INSERT Input)

The 'Create New Campus Event' modal dialog populated with new event specifications: Event Title ('Inter-College Sports 2026'), Academic Category ('Sports & Athletics'), Badge Tag ('Event'), Venue Location ('Main Library, Hall H'), Date & Time ('10/29/2026 08:30 AM'), Seating Capacity ('50'), and Status ('Upcoming').



2.4 Output 4: Event Poster File Selection & Dropzone Readiness

The drag-and-drop file upload dropzone within the creation modal. The administrator attaches a valid event poster: 'Sports.jpg (73 KB) • Ready', verifying client-side file reading, size validation (< 2MB limit), and thumbnail preview generation prior to server dispatch.



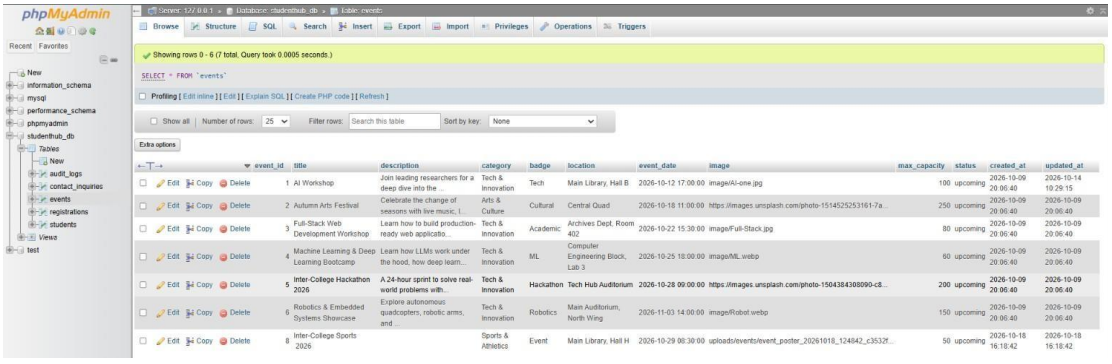
2.5 Output 5: Event Creation & Poster Upload Success Notification

High-contrast green toast feedback notification rendered upon successful backend processing: 'Event 'Inter-College Sports 2026' successfully published and registered in campus registry.', confirming that the poster binary was uploaded to disk and database record was committed.



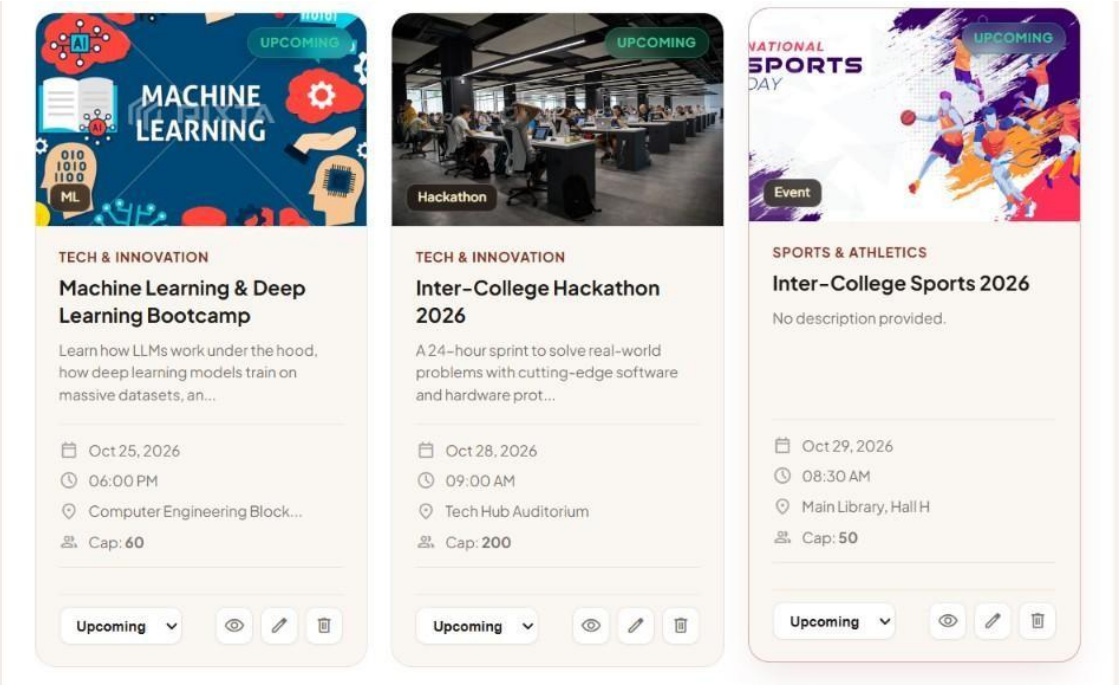
2.6 Output 6: MySQL Database Verification of Inserted Event Record

phpMyAdmin browse view of the 'events' table verifying that event_id 8 ('Inter-College Sports 2026') has been successfully inserted with category 'Sports & Athletics', capacity '50', and the uniquely sanitized poster image path: 'uploads/events/event_poster_20261018_124842_c3532f...'.



2.7 Output 7: Admin Event Moderation Center Live Grid Rendering

The updated Event Moderation board on the administrative console rendering the newly published 'Inter-College Sports 2026' event card alongside existing events. The card prominently displays the uploaded sports poster, scheduled timing ('Oct 29, 2026 | 08:30 AM'), venue ('Main Library, Hall H'), and capacity ('Cap: 50').



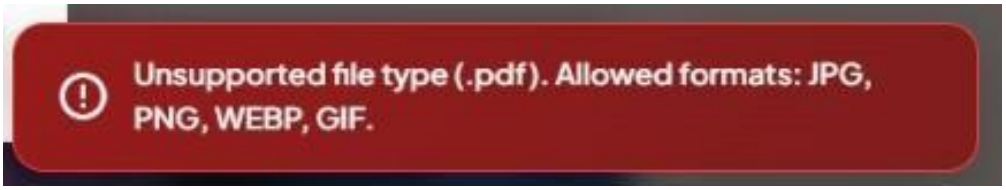
2.8 Output 8: Cross-Portal Synchronization — Student Catalog View

The public Student Event Discovery catalog ('events.html') instantly synchronizing with the database update. The newly added event 'Inter-College Sports 2026' appears at the top of the student exploration grid with the uploaded banner image and an active 'Register Now' booking trigger, proving real-time architectural linkage.



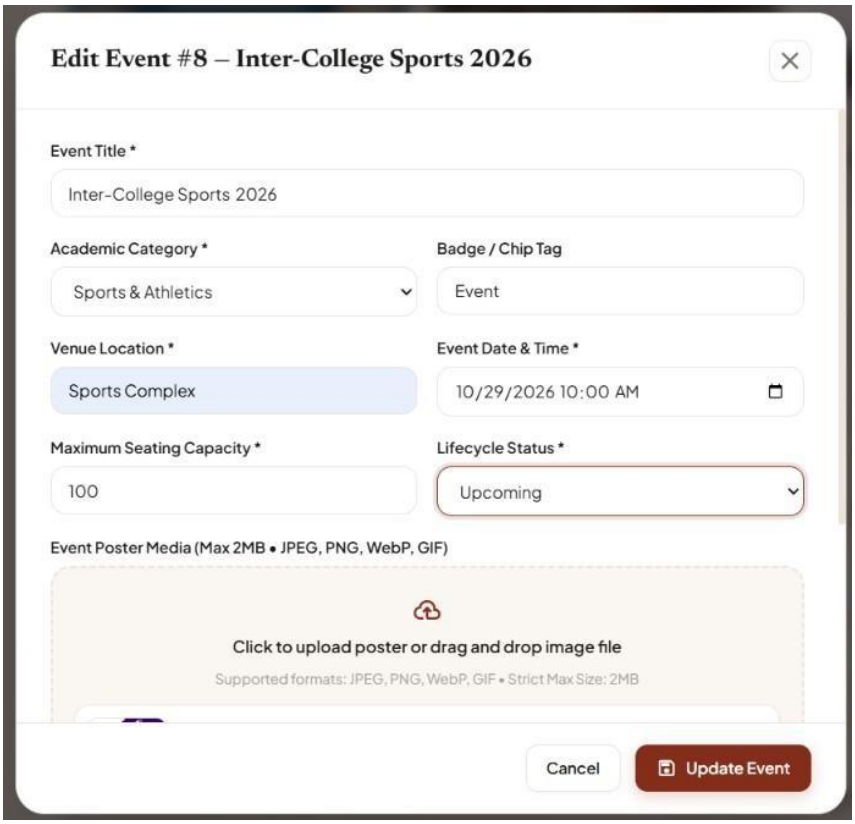
2.9 Output 9: Security Validation Guard — Unsupported File Type Rejection

Defensive server-side and client-side file validation in action. When an administrator attempts to upload an unsupported file format (e.g., '.pdf'), the system blocks the upload and triggers an immediate high-contrast red error toast: 'Unsupported file type (.pdf). Allowed formats: JPG, PNG, WEBP, GIF.', preventing arbitrary file execution.



2.10 Output 10: Edit Campus Event Interface (UPDATE Input)

The 'Edit Event #8 – Inter-College Sports 2026' modification modal allowing administrators to adjust event parameters. The administrator updates the venue location to 'Sports Complex', modifies the schedule to '10/29/2026 10:00 AM', and expands maximum seating capacity from 50 to '100'.



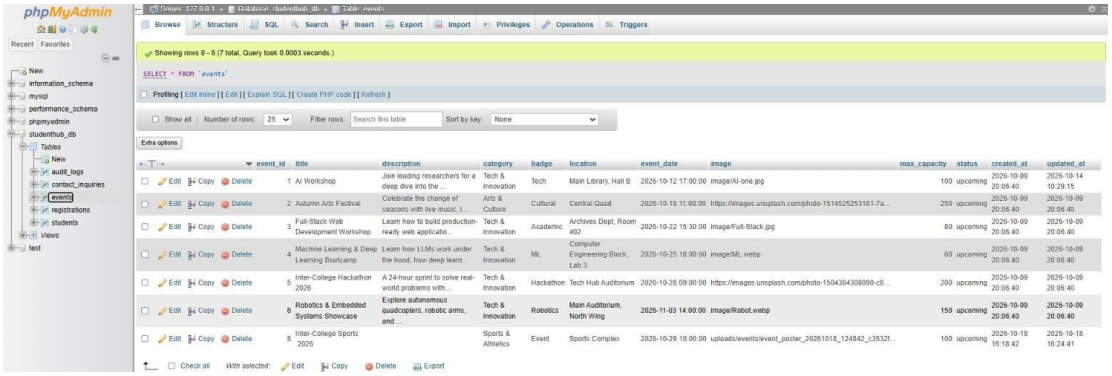
2.11 Output 11: Event Update Success Notification

Real-time UI feedback banner confirming successful event attribute modification: 'Event 'Inter-College Sports 2026' updated successfully.', verifying that the server-side UPDATE statement was executed cleanly.



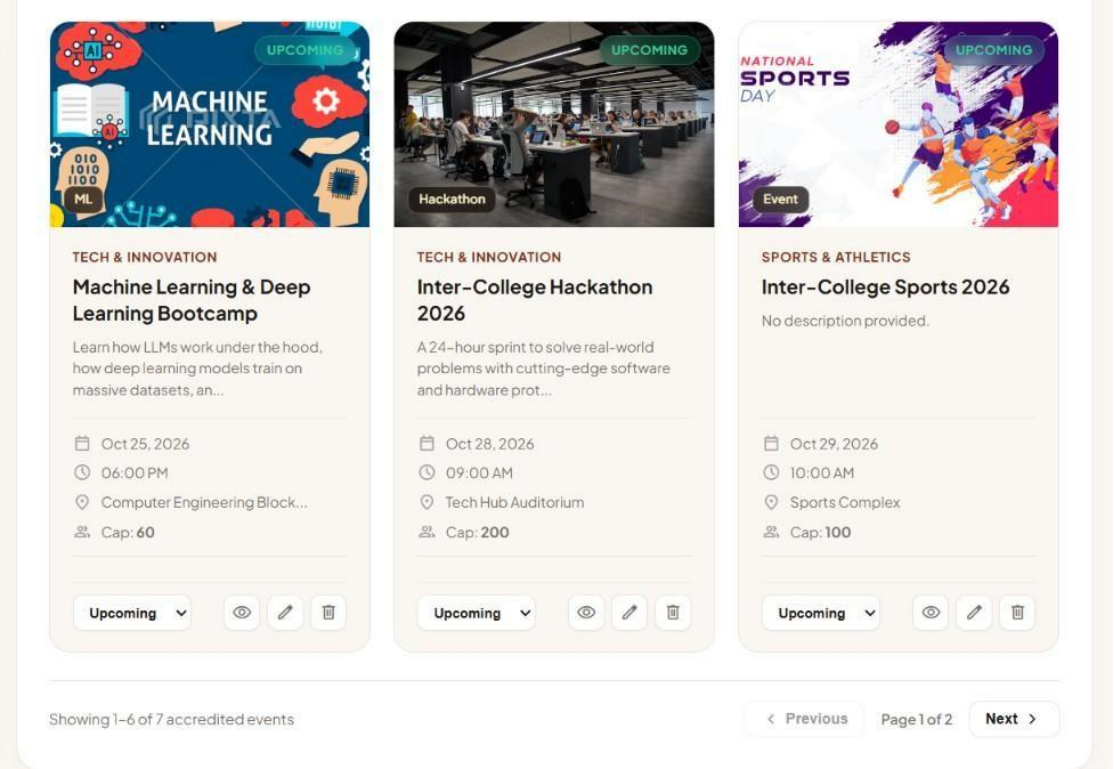
2.12 Output 12: MySQL Database Verification of Updated Record

phpMyAdmin browse view of the 'events' table confirming that event_id 8 reflects the updated venue location ('Sports Complex'), modified event_date ('2026-10-29 10:00:00'), expanded capacity ('100'), and automatic update of the 'updated_at' timestamp ('2026-10-18 16:24:41').



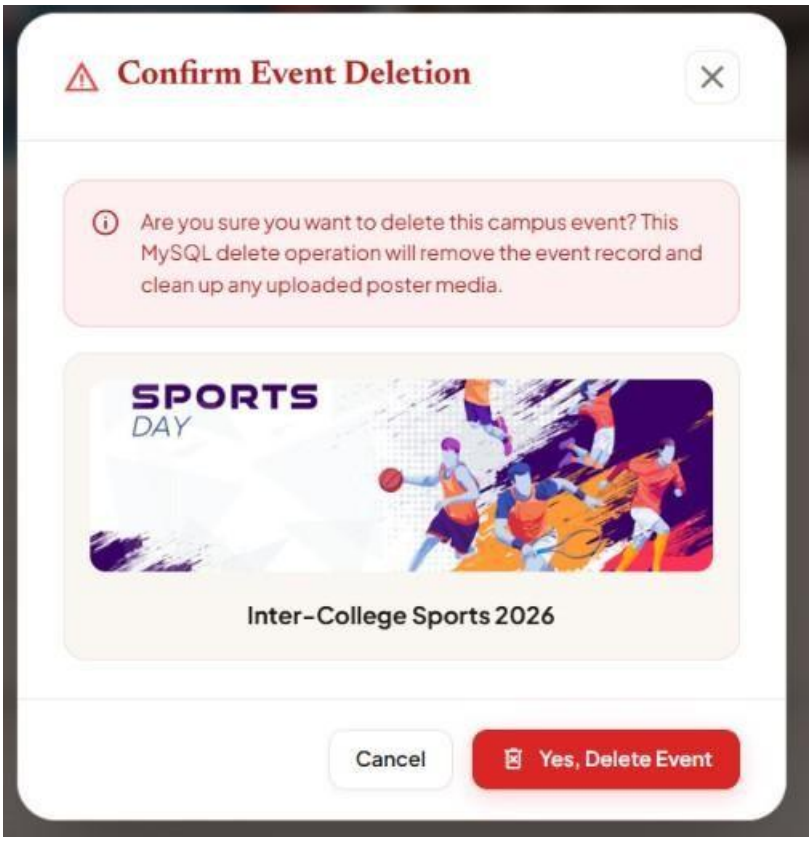
2.13 Output 13: Admin Dashboard Updated Event Card Display

The live Event Moderation board dynamically reflecting the updated event card details: scheduled time 'Oct 29, 2026 | 10:00 AM', venue 'Sports Complex', and updated seating capacity 'Cap: 100', verifying immediate DOM synchronization.



2.14 Output 14: Defensive Event Deletion Confirmation Guard Modal

The defensive 'Confirm Event Deletion' modal dialog triggered upon clicking the delete action icon. The dialog renders a preview banner of 'Inter-College Sports 2026' with an explicit warning: 'Are you sure you want to delete this campus event? This MySQL delete operation will remove the event record and clean up any uploaded poster media.' with 'Cancel' and 'Yes, Delete Event' action triggers.



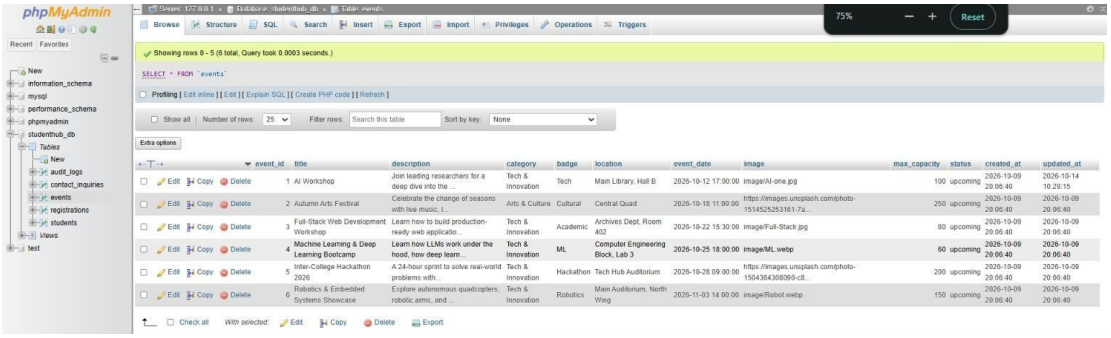
2.15 Output 15: Event Deletion Success Notification

System confirmation alert rendered following confirmed event removal: 'Event 'Inter-College Sports 2026' permanently removed from campus registry.', letting the administrator know that the database record was expunged and media assets unlinked.



2.16 Output 16: MySQL Database Verification of Permanent Event Removal

phpMyAdmin database browse view of the 'events' table confirming that event_id 8 has been permanently deleted from the database, leaving exactly the 6 baseline campus events and proving referential integrity.



3. Key Questions & Technical Analysis

Q1. Are add, update, and delete functionalities implemented correctly?

Yes. All CRUD operations were rigorously implemented using parameterized MySQL statements and verified against both database tables and UI views:

- **Add (INSERT):** The backend processes text inputs and $_FILES['poster']. After validation, it moves the uploaded image to 'uploads/events/' using move_uploaded_file() and executes an INSERT INTO events (title, description, category, badge, location, event_date, image, max_capacity, status) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?).
- Update (UPDATE): Administrators can edit text fields with or without uploading a replacement poster. If a new image is provided, the old poster is unlinked from the server and the new path is saSmit; otherwise, the existing image path is retained.
- **Delete (DELETE):** Deleting an event triggers a database query DELETE FROM events WHERE event_id = ?. Due to foreign key constraints, related event registrations in the 'registrations' table are safely cascaded, and the physical poster file on disk is deleted using unlink().

Q2. Is the UI linked with the database correctly?

Yes. The interface is dynamically linked to MySQL across the full application ecosystem:

- **Two-Way Data Binding:** The admin event grid is populated directly from database records via asynchronous Fetch requests. When an event is added or modified in the database, the grid updates immediately without page reloads.
- **Cross-Portal Linkage:** Events committed to the database instantly become available on the student catalog ('events.html'). Verified outputs show that 'Inter-College Sports 2026' was immediately accessible for student RSVPs.

Q3. Are poster uploads validated for type and size?

Yes. Multi-layer file validation protects against corrupted, oversized, or malicious file submissions:

- **MIME Type & Extension Whitelist:** The backend checks both the file extension (pathinfo($filename, PATHINFO_EXTENSION)) and the binary MIME type via finfo_file(). Allowed types are strictly constrained to image/jpeg, image/png, image/webp, and image/gif. Unsupported files (like .pdf) trigger immediate rejection.
- **File Size Enforcement:** The $_FILES['poster']['size'] attribute is validated against a 2MB limit ($max_size = 2 * 1024 * 1024). Submissions exceeding this limit are terminated before reaching storage.

Q4. Are success and failure messages displayed?

Yes. Every user action generates explicit visual feedback:

- Success Messages: High-contrast green toast alerts confirm successful creation ('Event published successfully'), updates ('Event updated successfully'), and deletion ('Event permanently removed').
- **Error Messages:** Actionable red banners alert administrators when validation fails, such as the verified output: 'Unsupported file type (.pdf). Allowed formats: JPG, PNG, WEBP, GIF.'

4. GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

5. Conclusion & Learning Outcomes

Through the execution of Practical 12, a secure, feature-complete Event Management CRUD module with binary poster upload capabilities was successfully developed, verified, and integrated into the StudentHub University Portal. The practical demonstrated end-to-end integration between frontend multipart/form-data forms and PHP server-side media processing. Robust file validation algorithms—including MIME type binary inspection, extension whitelisting, strict 2MB size enforcement, and cryptographically safe filename randomization—were implemented to protect the server against arbitrary file upload exploits and path traversal attacks. Full CRUD lifecycle operations were verified through live administrative event creation, database persistence in phpMyAdmin, real-time synchronization with the public student event catalog, attribute updating, and defensive event deletion with modal confirmation guards. This practical fulfills Course Outcomes CO1, CO4, and CO5 by reinforcing essential skills in binary media handling, database-driven administrative portals, and secure web application development.